Lab 8: Controlling and Reading Volatile Sensors

11/05/2026

Daniel Xu
ECE 3218L

1 Overview

The Pololu 3pi+ 32U4 has five down-facing IR reflectance sensors. Each sensor is read by charging a 2.2 nF capacitor and timing how long it takes to discharge through a phototransistor: a bright surface reflects more IR, so the capacitor discharges faster. In this lab, all five sensors were read in parallel with Timer3, their discharge times were measured over white and black surfaces, and the readings were used to estimate the robot's angle to a 1 cm black line.

2 Procedure

- **Setup** Table 1 lists the pins, taken from the Pololu 3pi+ 32U4 User's Guide. PF4, PF5 and PF7 are also JTAG pins, so JTAG is disabled in software (the JTD bit in MCUCR, written twice within 4 cycles) before they are used. Timer3 runs in normal mode with prescaler $N = 8$, so each tick is 8/16 MHz = 0.5 μs. Timer1 is left free for the motors from Lab 7.

Table 1: Pins used in this lab.

Pin	Function	Mode
PD6	DN1 (left)	Output to charge, input to time
PF7	DN2	Output to charge, input to time
PF5	DN3 (center)	Output to charge, input to time
PF4	DN4	Output to charge, input to time
PF1	DN5 (right)	Output to charge, input to time
PB7	IR emitter control	High = line emitters on, input = off
PB3	Button A	Input, pull-up

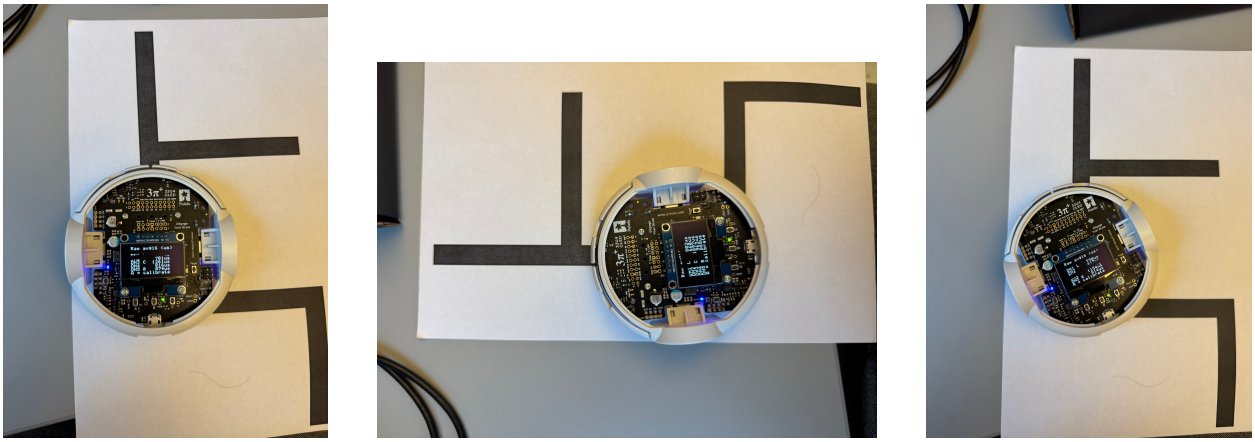
- **Reading the sensors** One reading of all five sensors works as follows (Code 3):

  1. Turn the IR emitters on by driving PB7 high.
  2. Charge the capacitors by making every sensor pin an output and driving it high for 10 μs. The charging time constant is 220 $\Omega \times 2.2$ nF ≈ 0.5 μs, so 10 μs fully charges them.
  3. Reset TCNT3 and release the pins: clear DDR first, then PORT. Clearing PORT first would drive the pins low and drain the capacitors. Clearing PORT afterwards turns the pull-ups off, which would otherwise keep feeding the capacitors.
  4. Poll the pins. When a pin reads 0, record TCNT3 as that sensor's time.

5. Stop when all five pins are low or after 20000 ticks (10 ms). Any sensor still high is given the timeout value.
6. Turn the emitters off by making PB7 an input. Driving PB7 low would turn on the bump sensor emitters instead.

The five pin levels are packed into one 5-bit number each time through the polling loop (bit i = sensor i). A second 5-bit number, `pending`, marks the sensors still waiting to go low, so the sensors that just went low are `pending & ~levels`. Each one gets the current timer value and is cleared from `pending`, so it is only recorded once.

- **Program** Button A moves through three screens (Code 1). The first shows each sensor's discharge time averaged over 16 readings, in μs, which was used for every measurement in this lab. The second and third screens (calibration and line position) were written for the line-following labs.
- **Experiment 1: bright and dark surfaces** The robot was set on white paper and read five times, moving it slightly between readings. This was repeated on a black surface (the black lining of the robot's box).
- **Experiment 2: angle to a line** The robot was centered on a 1 cm black line printed on white paper, first straight along it, then turned 5° and 15° each way (Figure 1).



(a) Straight (b) 5° (photo rotated) (c) 15°

Figure 1: Robot placements for the angle experiment.

3 Results

3.1 Bright and Dark Surfaces

Table 2: Discharge times on white paper, each an average of 16 readings (μs).

Sensor	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
DN1 (left)	780	772	782	782	797	783
DN2	600	601	593	596	606	599
DN3 (center)	533	532	530	538	526	532
DN4	702	702	705	765	691	713
DN5 (right)	858	854	853	870	864	860

Table 3: Discharge times on the black surface (μs). Dashes are readings that could not be read from the photos.

Sensor	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average
DN1 (left)	2048	2065	–	1959	2041	2028
DN2	2150	2146	2165	2114	2112	2137
DN3 (center)	2038	1910	1947	1976	1957	1966
DN4	2540	2502	2661	–	2555	2565
DN5 (right)	2196	2207	2252	–	2260	2229

Table 4: Contrast between the white and black averages.

Sensor	White (μs)	Black (μs)	Black − white	Black / white
DN1 (left)	783	2028	1246	2.59
DN2	599	2137	1538	3.57
DN3 (center)	532	1966	1434	3.70
DN4	713	2565	1852	3.60
DN5 (right)	860	2229	1369	2.59

3.2 Angle to a Line

Each reading t_i was converted to a darkness from 0 (white) to 1 (black) using that sensor's averages from Table 4, and the line's position x across the sensors was taken as the darkness-weighted average of the sensor positions p_i:

$$d_i = \frac{t_i - W_i}{B_i - W_i}, \qquad x = \frac{\sum_i d_i\, p_i}{\sum_i d_i}, \qquad p_i = -30, -10, 0, +10, +30 \text{ mm}. \tag{1}$$

The sensor positions come from Pololu's control board drawing: DN2–DN4 are 10 mm apart, and DN1 and DN5 are 30 mm from the center and sit about 12 mm behind the other three. Positive x means the line is to the right of center.

Table 5: Readings with the robot on the line (μs), darkness d_i, and line position x.

Placement	DN1	DN2	DN3	DN4	DN5	x (mm)
Straight	–	781	1261	816	874	−0.5
d_i	–	0.12	0.51	0.06	0.01	
5°	802	660	1097	963	868	+1.1
d_i	0.02	0.04	0.39	0.14	0.01	
5°, other way	–	–	1015	798	876	–
15°, both ways	not readable					–

The values were read from photos of the OLED. Several photos were taken while the screen was being redrawn, so some rows were cut off or shifted against their labels. Those values are marked with dashes. Both 15° readings were affected, so no numbers could be taken from them.

4 Post-Lab

Comment on any unexpected behaviors exhibited by the sensors during the experiments.

- **DN4 reads high.** DN4 was about 34% slower than DN3 on white and 30% slower on black. A second robot running the same code did not show this, so the difference is in this robot's hardware, not the program. Its black/white ratio (3.60) matches DN2 (3.57) and DN3 (3.70), so DN4 is not weak; every reading is scaled up by about the same factor. This is normal part-to-part variation in the phototransistor or capacitor, and it cancels out when each sensor is normalized with its own white and black averages (Equation 1).
- **The outer sensors have less contrast.** DN1 and DN5 have a black/white ratio of 2.59, against 3.57–3.70 for the inner three. Both outer sensors read slower on white, which points to their position: they sit about 12 mm behind the inner row, near the edge of the shell.
- **Black was nowhere near the timeout.** The black surface read about 2.0–2.6 ms, far below the 10 ms timeout, so black was only 2.6–3.7 times slower than white. The black box lining still reflects a fair amount of IR.
- **A sensor centered on the line reads only halfway to black.** With the robot straight, DN3 read 1261 μs ($d = 0.51$), even though the line was directly under it. Each sensor sees a spot wider than the 1 cm line, so it averages the line and the paper around it. DN2 also read slightly dark ($d = 0.12$).

Describe and comment on the results of part 3 above.
Experiment 1: every sensor's readings were repeatable within about ±2% on white, except one 765 μs reading on DN4 (9% above its other trials). White averaged 532–860 μs and black 1966–2565 μs. Every sensor separates white from black clearly, but each needs its own white and black values before the sensors can be compared.

Experiment 2: going from straight to 5° moved the line from $x = -0.5$ mm to $x = +1.1$ mm: DN3 dropped from 1261 to 1097 μs and DN4 rose from 816 to 963 μs. The second 5° placement, turned the other way, also lowered DN3 (1015 μs) while DN4 stayed near white (798 μs), and DN2 showed a four-digit value that was cut off in the photo, so the line most likely moved toward DN2. The darkened neighbor therefore shows which way the robot is turned.

One reading cannot show the angle directly. The line crosses the sensor row over a width of $1\ \text{cm}/\cos\theta$, which is 10.0, 10.04 and 10.35 mm at 0°, 5° and 15°, a difference far smaller than the 10 mm sensor spacing. What the readings do show is how far the line moved sideways: when the robot turns by θ about a point a distance L behind the sensors, the sensors swing sideways by $L\sin\theta$. The 5° trial moved x by 1.6 mm, about 0.3 mm per degree. That is less than expected for the sensors' distance from the robot's center, probably because each sensor's wide view averages the line over a large area (which flattens the weighted average) and because the robot was not turned exactly about its center. With the 15° readings lost, this scale could not be checked at a second angle. Taking a second reading after moving the robot a known distance D along the line, $\theta = \arctan(\Delta x/D)$, avoids both problems and gives the sign of the angle.

5 Discussion/Conclusion

All five 3pi+ reflectance sensors were read in parallel by charging their capacitors, releasing them together, and timing each discharge with Timer3. White surfaces read 532–860 μs and black 1966–2565 μs, with DN4 consistently slow and the outer sensors less sensitive, so each sensor needs its own white and black calibration. A single reading shows where the line is under the robot and which way the robot is turned, but the angle itself is better measured from the change in line position between two readings.

A Full Code

Code 1: Raw, calibration and line screens (main.c).

```
// ECE 3218 Lab 8: Controlling and Reading Volatile Sensors
//
// Three screens, each moved on from with button A:
//
//   1. Raw     Average discharge time of each sensor in us (Part 3.1).
//   2. Cal     Sweep the robot back and forth over the line so every
//              sensor sees both white and black. A = done.
//   3. Line    Darkness of each sensor (0-99), the line position x, and
//              the angle from two positions (Part 3.2): put the robot on
//              the line, press A to mark x, slide it SLIDE_MM straight
//              forward, and read the angle.
//
// Press RESET to start over.

#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "OLED.h"
#include "button.h"
#include "line.h"
#include "sensors.h"

#define RAW_SAMPLES   16   // readings averaged per update on screen 1
#define LINE_SAMPLES  4    // readings averaged per update on screen 3
#define CAL_REDRAW    8    // readings between redraws on screen 2

// Distance to slide the robot forward between the two angle readings
#define SLIDE_MM      30

static uint16_t sensor_values[SENSOR_COUNT];
static char text[OLED_TEXT_MAX + 1];

static const char *const sensor_names[SENSOR_COUNT] = {
    "DN1 L", "DN2  ", "DN3 C", "DN4  ", "DN5 R"
};

static void line_text(uint8_t line, const char *s)
{
    snprintf(text, sizeof text, "%s", s);
    OLED_writeTextLine(line, text);
}

// 1 once per press of button A
static uint8_t a_clicked(void)
{
    if (!button_a_pressed())
        return 0;
    button_a_wait_release();
    return 1;
```

```
}

// Tenths of a mm as "+12.5"
static void format_tenths(char *out, size_t size, int16_t v)
{
    uint16_t a = (uint16_t)abs(v);
    snprintf(out, size, "%c%u.%u", v < 0 ? '-' : '+', a / 10, a % 10);
}

// Darkness 0-1000 as 0-99, so five fit on one OLED line
static unsigned dark_99(uint16_t d)
{
    unsigned v = d / 10;
    return v > 99 ? 99 : v;
}

// Average of n readings, in ticks
static void read_average(uint16_t avg[SENSOR_COUNT], uint8_t n)
{
    uint32_t sum[SENSOR_COUNT] = { 0 };

    for (uint8_t k = 0; k < n; k++) {
        sensors_read(sensor_values);
        for (uint8_t i = 0; i < SENSOR_COUNT; i++)
            sum[i] += sensor_values[i];
    }
    for (uint8_t i = 0; i < SENSOR_COUNT; i++)
        avg[i] = (uint16_t)(sum[i] / n);
}

// ---- Screen 1: raw times ---------------------------------------------------
static void raw_screen(void)
{
    uint16_t avg[SENSOR_COUNT];

    OLED_clearDisplayRam();
    snprintf(text, sizeof text, "Raw avg%u (us)", RAW_SAMPLES);
    OLED_writeTextLine(0, text);
    line_text(7, "A = calibrate");

    while (!a_clicked()) {
        read_average(avg, RAW_SAMPLES);
        for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
            snprintf(text, sizeof text, "%s %5uus", sensor_names[i],
                     avg[i] / SENSOR_TICKS_PER_US);
            OLED_writeTextLine(i + 2, text);
        }
    }
}

// ---- Screen 2: calibration -------------------------------------------------
static void calibrate_screen(line_cal_t *cal)
{
    OLED_clearDisplayRam();
    line_text(0, "Sweep over line");
    line_text(1, "  min    max us");
    line_text(7, "A = done");
    line_cal_reset(cal);

```

```
    for (uint8_t n = 0; ; n++) {
        sensors_read(sensor_values);
        line_cal_update(cal, sensor_values);

        if (n % CAL_REDRAW == 0) {
            for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
                snprintf(text, sizeof text, "%u %5u %5u", i + 1,
                         cal->min[i] / SENSOR_TICKS_PER_US,
                         cal->max[i] / SENSOR_TICKS_PER_US);
                OLED_writeTextLine(i + 2, text);
            }
        }

        if (a_clicked()) {
            int8_t bad = line_cal_check(cal);
            if (bad < 0)
                return;
            snprintf(text, sizeof text, "DN%d not swept!", bad + 1);
            OLED_writeTextLine(7, text);
        }
    }
}

// ---- Screen 3: line position and angle -----------------------------------
//
// x is where the line is under the sensors (+ = right of center). If x
// changes by dx while the robot slides SLIDE_MM straight forward, the
// robot's heading is atan(dx / SLIDE_MM) off the line.
// A positive angle means the robot is turned left (CCW) of the line.
static void line_screen(const line_cal_t *cal)
{
    uint16_t avg[SENSOR_COUNT];
    uint16_t dark[SENSOR_COUNT];
    int16_t  x = 0, x_mark = 0;
    uint8_t  marked = 0;
    char     num[8];

    OLED_clearDisplayRam();
    line_text(0, "Line  A = mark x");
    snprintf(text, sizeof text, "slide %umm fwd", SLIDE_MM);
    OLED_writeTextLine(7, text);

    for (;;) {
        read_average(avg, LINE_SAMPLES);
        line_normalize(cal, avg, dark);
        uint8_t found = line_position(dark, &x);

        snprintf(text, sizeof text, "d %2u %2u %2u %2u %2u",
                 dark_99(dark[0]), dark_99(dark[1]), dark_99(dark[2]),
                 dark_99(dark[3]), dark_99(dark[4]));
        OLED_writeTextLine(2, text);

        if (found) {
            format_tenths(num, sizeof num, x);
            snprintf(text, sizeof text, "x  %smm", num);
        } else {
            snprintf(text, sizeof text, "x  no line");
        }
        OLED_writeTextLine(3, text);
```

```

        if (a_clicked() && found) {
            x_mark = x;
            marked = 1;
        }

        if (marked && found) {
            int16_t dx = x - x_mark;
            int16_t deg = (int16_t)lround(atan2(dx / 10.0, SLIDE_MM)
                                          * 180.0 / M_PI);
            format_tenths(num, sizeof num, dx);
            snprintf(text, sizeof text, "dx %smm", num);
            OLED_writeTextLine(5, text);
            snprintf(text, sizeof text, "ang %+d deg", deg);
            OLED_writeTextLine(6, text);
        } else {
            line_text(5, marked ? "dx no line" : "dx (press A)");
            line_text(6, "");
        }
    }
}

int main(void)
{
    line_cal_t cal;

    OLED_init();
    OLED_reset();
    OLED_clearDisplayRam();
    OLED_configureDefault();
    sensors_init();
    button_a_init();

    raw_screen();
    calibrate_screen(&cal);
    line_screen(&cal);
}
```

Code 2: Sensor interface and pin map (sensors.h).

```
// Down-facing IR reflectance sensors on the Pololu 3pi+ 32U4, left to right:
//
//   index  name          pin
//     0    DN1 (left)    PD6
//     1    DN2           PF7
//     2    DN3 (center)  PF5
//     3    DN4           PF4   (the handout's table calls this one "DN5")
//     4    DN5 (right)   PF1
//
// Emitter control is PB7: high = line sensor emitters on, low = bump sensor
// emitters on, input = all emitters off.
//
// A reading is how long the sensor's capacitor takes to discharge.
// Bright surface = small number, dark surface = large number.
// Timed with Timer3 at 16 MHz / 8, so 1 tick = 0.5 us.

#ifndef SENSORS_H
#define SENSORS_H
```

```

#include <stdint.h>

#define SENSOR_COUNT         5
#define SENSOR_TICKS_PER_US  2

// Sensors still high after this many ticks read as exactly this value.
// 20000 ticks = 10 ms, as in the handout example.
#define SENSOR_TIMEOUT       20000u

// Set up the emitter pin, the sensor pins and Timer3. Call once.
void sensors_init(void);

// Read all five sensors at once. values[i] is the discharge time of
// sensor i in ticks. Takes at most SENSOR_TIMEOUT ticks.
void sensors_read(uint16_t values[SENSOR_COUNT]);

#endif // SENSORS_H
```

Code 3: Parallel sensor reading with Timer3 (sensors.c).

```
// All five sensors are charged, released and polled together. Each pass of
// the polling loop packs the five pin levels into one 5-bit number
// (bit i = sensor i), so the sensors that just went low are
//
//     fell = pending & ~levels;
//
// Each one gets the current timer value and is removed from ‘pending‘.

#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <util/delay.h>
#include "sensors.h"

#define EMIT  PB7

#define DN1   PD6
#define DN2   PF7
#define DN3   PF5
#define DN4   PF4
#define DN5   PF1

// Sensor pins grouped by port, so each port is switched in one write
#define SENSOR_MASK_D  (1 << DN1)
#define SENSOR_MASK_F  ((1 << DN2) | (1 << DN3) | (1 << DN4) | (1 << DN5))

#define ALL_SENSORS    ((1 << SENSOR_COUNT) - 1)

#define CHARGE_US      10   // 220 ohm * 2.2 nF = 0.5 us, so 10 us is plenty

// Current level of every sensor pin, packed into bits 0..4
static inline uint8_t sensor_levels(void)
{
    uint8_t d = PIND;
    uint8_t f = PINF;
```

```
38
39      return (((d >> DN1) & 1) << 0)
40           | (((f >> DN2) & 1) << 1)
41           | (((f >> DN3) & 1) << 2)
42           | (((f >> DN4) & 1) << 3)
43           | (((f >> DN5) & 1) << 4);
44  }
45
46  // Driving PB7 low would turn on the bump sensor emitters, so "off" means
47  // making PB7 an input with no pull-up.
48  static void emitters_off(void)
49  {
50      DDRB  &= ~(1 << EMIT);
51      PORTB &= ~(1 << EMIT);
52  }
53
54  static void emitters_on(void)
55  {
56      PORTB |= (1 << EMIT);
57      DDRB  |= (1 << EMIT);
58  }
59
60  void sensors_init(void)
61  {
62      // PF4, PF5 and PF7 are also JTAG pins and can't be used as I/O while
63      // JTAG is on. JTD must be written twice within 4 cycles, so compute
64      // the value first and then write it twice back to back.
65      uint8_t mcucr = MCUCR | (1 << JTD);
66      MCUCR = mcucr;
67      MCUCR = mcucr;
68
69      emitters_off();
70
71      // Sensor pins: inputs, pull-ups off
72      DDRD  &= ~SENSOR_MASK_D;
73      PORTD &= ~SENSOR_MASK_D;
74      DDRF  &= ~SENSOR_MASK_F;
75      PORTF &= ~SENSOR_MASK_F;
76
77      // Timer3: normal mode, clk/8 -> 0.5 us per tick
78      TCCR3A = 0x00;
79      TCCR3B = (1 << CS31);
80  }
81
82  void sensors_read(uint16_t values[SENSOR_COUNT])
83  {
84      emitters_on();
85
86      // Charge the capacitors: drive every sensor pin high
87      PORTD |= SENSOR_MASK_D;
88      PORTF |= SENSOR_MASK_F;
89      DDRD  |= SENSOR_MASK_D;
90      DDRF  |= SENSOR_MASK_F;
91      _delay_us(CHARGE_US);
92
93      // Release the pins and start timing. Clear DDR before PORT: clearing
94      // PORT first would drive the pins low and drain the capacitors.
95      // Clearing PORT afterwards turns the pull-ups off, which would
96      // otherwise keep the capacitors charged.
```

```
    TCNT3 = 0;
    DDRD  &= ~SENSOR_MASK_D;
    DDRF  &= ~SENSOR_MASK_F;
    PORTD &= ~SENSOR_MASK_D;
    PORTF &= ~SENSOR_MASK_F;

    // Poll until every sensor has gone low or we time out. The time is
    // read before the pins, so a recorded time is never later than the
    // real edge.
    uint8_t  pending = ALL_SENSORS;
    uint16_t now;

    do {
        now = TCNT3;
        uint8_t fell = pending & ~sensor_levels();

        if (fell) {
            for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
                if (fell & (1 << i))
                    values[i] = now;
            }
            pending &= ~fell;
        }
    } while (pending && now < SENSOR_TIMEOUT);

    // Anything still high timed out
    for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
        if (pending & (1 << i))
            values[i] = SENSOR_TIMEOUT;
    }

    emitters_off();
}
```

Code 4: Calibration and line position interface (line.h).

```
// Turns raw sensor times into "how dark" values and a line position.
//
// Sensor positions come from Pololu's 3pi+ 32U4 OLED control board drawing
// (bottom view): DN2-DN4 are 10 mm apart, and DN1/DN5 are 30 mm from the
// center and sit about 12 mm behind the other three. Check with a ruler.

#ifndef LINE_H
#define LINE_H

#include <stdint.h>
#include "sensors.h"

// Sideways position of each sensor from the center, in tenths of a mm
// (left is negative).
#define LINE_SENSOR_X_TENTHS_MM  { -300, -100, 0, 100, 300 }

// A sensor must change by at least this much (Timer3 ticks) between
// white and black during calibration, or calibration is rejected.
#define LINE_CAL_MIN_RANGE  500

// Fully dark reads as this value after normalizing.
#define LINE_DARK_MAX  1000
```

```h

// Below this total darkness there is no line under the sensors.
#define LINE_MIN_TOTAL  300

typedef struct {
    uint16_t min[SENSOR_COUNT];    // whitest time seen (ticks)
    uint16_t max[SENSOR_COUNT];    // darkest time seen (ticks)
} line_cal_t;

// Start a new calibration.
void line_cal_reset(line_cal_t *cal);

// Widen each sensor's min/max with one reading.
void line_cal_update(line_cal_t *cal, const uint16_t values[SENSOR_COUNT]);

// Index of the first sensor whose min/max range is too small (it never
// saw both white and black), or -1 if every sensor is calibrated.
int8_t line_cal_check(const line_cal_t *cal);

// Darkness of each sensor: 0 on white, LINE_DARK_MAX on black.
void line_normalize(const line_cal_t *cal, const uint16_t values[SENSOR_COUNT],
                    uint16_t dark[SENSOR_COUNT]);

// Darkness-weighted average of the sensor positions, in tenths of a mm
// (0 = line centered, negative = line to the left). Returns 0 and leaves
// *x unchanged if there is no line under the sensors.
uint8_t line_position(const uint16_t dark[SENSOR_COUNT], int16_t *x);

#endif // LINE_H
```

Code 5: Calibration and line position (line.c).

```c
#include "line.h"

static const int16_t sensor_x[SENSOR_COUNT] = LINE_SENSOR_X_TENTHS_MM;

void line_cal_reset(line_cal_t *cal)
{
    for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
        cal->min[i] = UINT16_MAX;
        cal->max[i] = 0;
    }
}

void line_cal_update(line_cal_t *cal, const uint16_t values[SENSOR_COUNT])
{
    for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
        if (values[i] < cal->min[i]) cal->min[i] = values[i];
        if (values[i] > cal->max[i]) cal->max[i] = values[i];
    }
}

int8_t line_cal_check(const line_cal_t *cal)
{
    for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
        if (cal->max[i] < cal->min[i] ||
            cal->max[i] - cal->min[i] < LINE_CAL_MIN_RANGE)
            return (int8_t)i;
```

```
    }
    return -1;
}

void line_normalize(const line_cal_t *cal, const uint16_t values[SENSOR_COUNT],
                    uint16_t dark[SENSOR_COUNT])
{
    for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
        uint16_t lo = cal->min[i];
        uint16_t hi = cal->max[i];

        if (values[i] <= lo)
            dark[i] = 0;
        else if (values[i] >= hi)
            dark[i] = LINE_DARK_MAX;
        else
            dark[i] = (uint16_t)((uint32_t)(values[i] - lo) * LINE_DARK_MAX
                                 / (hi - lo));
    }
}

uint8_t line_position(const uint16_t dark[SENSOR_COUNT], int16_t *x)
{
    int32_t  weighted = 0;
    uint16_t total = 0;

    for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
        weighted += (int32_t)dark[i] * sensor_x[i];
        total    += dark[i];
    }

    if (total < LINE_MIN_TOTAL)
        return 0;

    *x = (int16_t)(weighted / total);
    return 1;
}
```

Code 6: Button A interface (button.h).

```
// Button A is on PB3. It connects the pin to ground when pressed, so we
// turn on the ATmega32U4's internal pull-up: released = HIGH, pressed = LOW.

#ifndef BUTTON_H
#define BUTTON_H

#include <stdint.h>

// Make PB3 an input with its pull-up enabled.
void button_a_init(void);

// 1 if button A is held down right now, 0 if not.
uint8_t button_a_pressed(void);

// Wait for a full press and release of button A (debounced).
void button_a_wait(void);

// Wait until button A is released (debounced). Call this after
```

```
// button_a_pressed() returns 1 so one press is only counted once.
void button_a_wait_release(void);

#endif // BUTTON_H
```

Code 7: Button A input with debouncing (button.c).

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <util/delay.h>
#include "button.h"

#define BTN_A  PB3

void button_a_init(void)
{
    DDRB  &= ~(1 << BTN_A);   // input
    PORTB |=  (1 << BTN_A);   // enable internal pull-up
    _delay_us(10);            // let the pin settle high
}

uint8_t button_a_pressed(void)
{
    return (PINB & (1 << BTN_A)) == 0;   // LOW = pressed
}

void button_a_wait(void)
{
    do {
        while (!button_a_pressed())
            ;                    // loop indefinitely until pressed
        _delay_ms(20);           // debounce
    } while (!button_a_pressed());

    button_a_wait_release();
}

void button_a_wait_release(void)
{
    do {
        while (button_a_pressed())
            ;
        _delay_ms(20);           // debounce
    } while (button_a_pressed());
}
```

Code 8: Course-provided OLED driver header, with OLED_writeTextLine() added (OLED.h).

```
/*
 * OLED.h
 *
 * Created: 9/19/2024 7:12:26 PM
 *  Author: sternbal
 */

```

```

#ifndef OLED_H_
#define OLED_H_

#define OLEDCLK PD3 // D1
#define OLEDRES PD2 // D0
#define OLEDDC PB0 // D17
#define OLEDMOSI PD5
// Characters per line of OLED_writeText(): text starts at column 16 and
// each character is 7 pixels wide (5 glyph + 2 gap), so 16 fit in 128.
#define OLED_TEXT_MAX 16
void OLED_clk_low(void);
void OLED_clk_high(void);
void OLED_mos_pin(uint8_t);
void OLED_write(uint8_t);
void OLED_commandMode(void);
void OLED_dataMode(void);
void OLED_reset(void);
void OLED_clearDisplayRam(void);
void OLED_configureDefault(void);
void OLED_writeText(char*);
void OLED_writeTextLine(uint8_t, char*);
#define OLED_TEXT_LINES 8
void OLED_init(void);
uint8_t getGlyphColumn(uint8_t, uint8_t);
#define SET_COL_ADDR_LOW 0x00
#define SET_COL_ADDR_HIGH 0x10
#define SET_CONTRAST 0x81
#define SET_SEG_REMAP 0xA0
#define SET_INV_DISP 0xA6
#define SET_DISPLAY_ON 0xAE
#define SET_PAGE_ADDR 0xB0
#define SET_COM_SCAN_DIR 0xC0

#endif /* OLED_H_ */
```

Code 9: Course-provided OLED driver, with OLED_writeTextLine() added (OLED.c).

```
/*
 * OLED.c
 *
 * Created: 9/19/2024 9:50:58 PM
 *  Author: sternbal
 */
#include <avr/io.h>
#include "OLED.h"
#define F_CPU 16000000L
#include <util/delay.h>
#include <avr/pgmspace.h>

extern const PROGMEM uint8_t pololuOledFont[][5];


void OLED_init(void)
{
    // set reset pin to output
    DDRD |= 1<<OLEDRES;

```

```
    // set clock pin to output and make it low
    DDRD |= 1<<OLEDCLK;
    PORTD &= ~(1<<OLEDCLK);

    // set MOSI pin to output
    DDRD |= 1<<OLEDMOSI;


    // set data/command pin to output
    DDRB |= 1<<OLEDDC;

}

void OLED_clk_low(void)
{
    PORTD &= ~(1<<OLEDCLK);

}
void OLED_clk_high(void)
{
    PORTD |= (1<<OLEDCLK);

}

void OLED_mos_pin(uint8_t val)
{
    if(val==0) PORTD &= ~(1<<OLEDMOSI);
    else PORTD |= 1<<OLEDMOSI;
}

void OLED_reset(void)
{
    PORTD &= ~(1<<OLEDRES);
    _delay_us(10);
    PORTD |= (1<<OLEDRES);
    _delay_us(10);
}


void OLED_clearDisplayRam(void)
{
    OLED_clk_low();
    OLED_commandMode();
    OLED_write(SET_COL_ADDR_LOW | 2);
    for (uint8_t page = 0;page < 8; page++)
    {
        OLED_commandMode();
        OLED_write(SET_PAGE_ADDR | page);
        OLED_write(SET_COL_ADDR_HIGH | 0);
        OLED_dataMode();
        for (uint8_t i = 0 ; i < 128; i++)
        {
            OLED_write(0);
        }
    }
}

void OLED_configureDefault(void)
{
```

```
80      OLED_clk_low();
81      OLED_commandMode();
82      OLED_write(SET_SEG_REMAP | 1);
83      OLED_write(SET_COM_SCAN_DIR | 8);
84      OLED_write(SET_CONTRAST);
85      OLED_write(0xFF);
86      OLED_write(SET_DISPLAY_ON | 1);
87  }
88
89  void OLED_commandMode(void)
90  {
91      PORTB &= ~(1<<OLEDDC);
92  }
93
94  void OLED_dataMode(void)
95  {
96      PORTB |= (1<<OLEDDC);
97  }
98
99
100 void OLED_write(uint8_t d)
101 {
102     OLED_clk_low();
103     OLED_mos_pin(d >> 7 & 0x01);
104     OLED_clk_high();
105
106     OLED_clk_low();
107     OLED_mos_pin(d >> 6 & 0x01);
108     OLED_clk_high();
109
110     OLED_clk_low();
111     OLED_mos_pin(d >> 5 & 0x01);
112     OLED_clk_high();
113
114     OLED_clk_low();
115     OLED_mos_pin(d >> 4 & 0x01);
116     OLED_clk_high();
117
118     OLED_clk_low();
119     OLED_mos_pin(d >> 3 & 0x01);
120     OLED_clk_high();
121
122     OLED_clk_low();
123     OLED_mos_pin(d >> 2 & 0x01);
124     OLED_clk_high();
125
126     OLED_clk_low();
127     OLED_mos_pin(d >> 1 & 0x01);
128     OLED_clk_high();
129
130     OLED_clk_low();
131     OLED_mos_pin(d >> 0 & 0x01);
132     OLED_clk_high();
133 }
134
135 void OLED_writeText(char *text)
136 {
137     OLED_writeTextLine(0, text);
138 }
```

```
139
140 // Same as OLED_writeText(), but on text line ‘line‘ (0-7, top to bottom).
141 // Each line is one 8-pixel-tall display page.
142 void OLED_writeTextLine(uint8_t line, char *text)
143 {
144     OLED_commandMode();
145     OLED_write(SET_PAGE_ADDR | (line & 0x07));
146     OLED_write(SET_COL_ADDR_HIGH | 1);
147     OLED_write(SET_COL_ADDR_LOW | 0);
148     OLED_dataMode();
149     // Draw the string up to its ’\0’, then fill the rest of the row with
150     // spaces so any leftovers from a longer previous message are erased.
151     for (uint8_t i=0;i<OLED_TEXT_MAX;i++)
152     {
153         char glyph=’ ’;
154         if (*text) glyph= *text++;
155
156         for (uint8_t pixelX=0;pixelX<5;pixelX++)
157         {
158             uint8_t column=getGlyphColumn(glyph,pixelX);
159             OLED_write(column);
160 //          OLED_write(column);
161         }
162         OLED_write(0);
163         OLED_write(0);
164     }
165 }
166 uint8_t getGlyphColumn(uint8_t glyph, uint8_t pixelX)
167 {
168     if(glyph>=0x20)
169     {
170         return pgm_read_byte(&pololuOledFont[glyph - 0x20][pixelX]);
171     }
172     return(0);
173 }
```